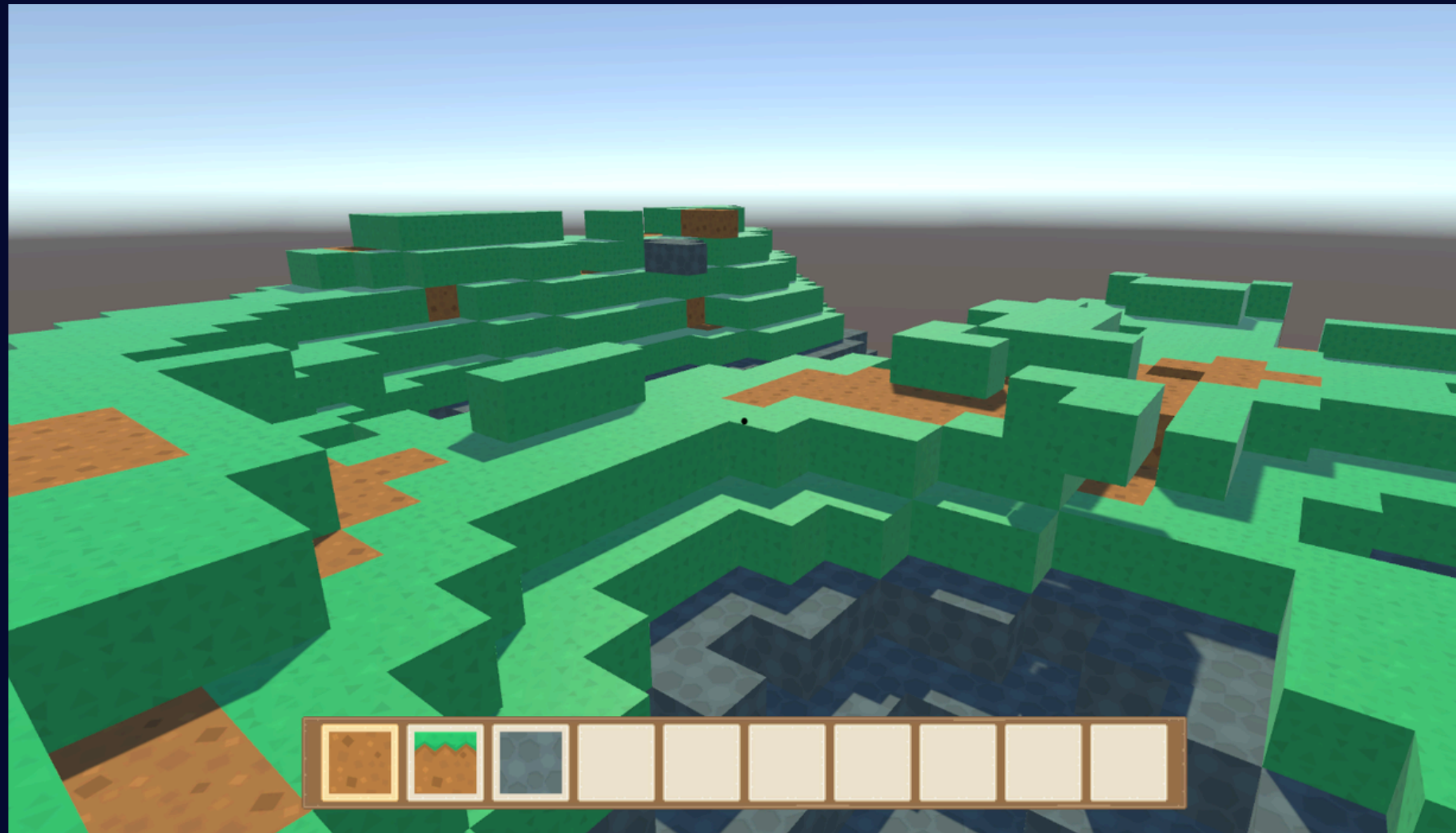


LogiCube



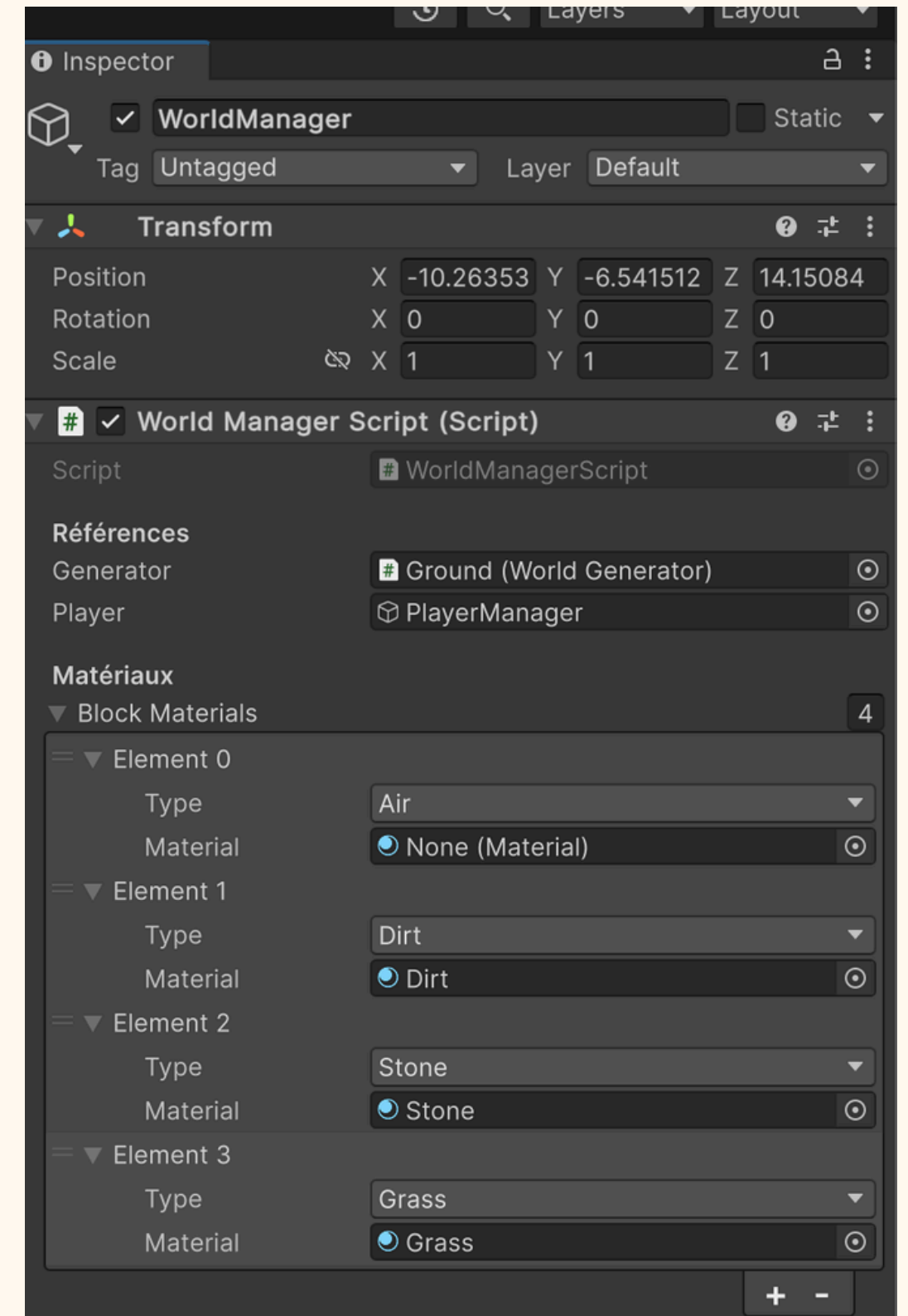
Inventaires, player et grass !

Première étape : ajouter un bloc d'herbe

Ajouter un bloc d'herbe

- Créez un nouveau matériau d'herbe ;
- Ajoutez Grass dans l'énumération BlockType ;
- Ajoutez ce matériau dans la liste Block Materials de WorldManager.

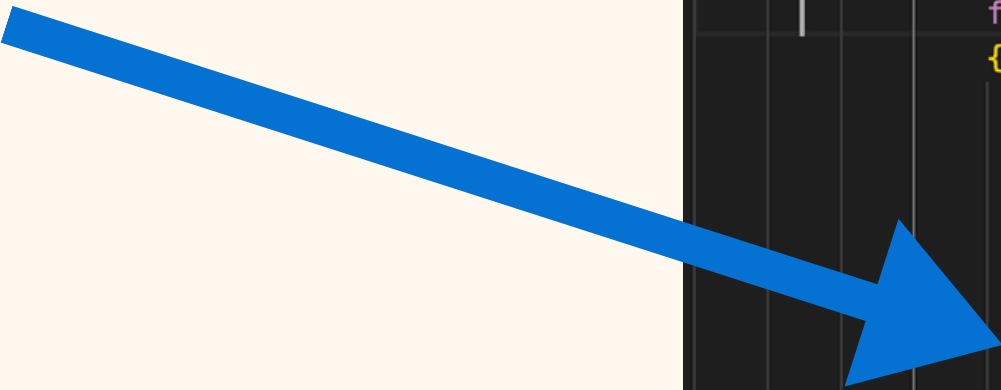
```
30 references
public enum BlockType
{
    1 reference
    Air = 0, // vide
    2 references
    Dirt = 1, // terre
    3 references
    Stone = 2, // pierre
    2 references
    Grass = 3, // herbe
}
```



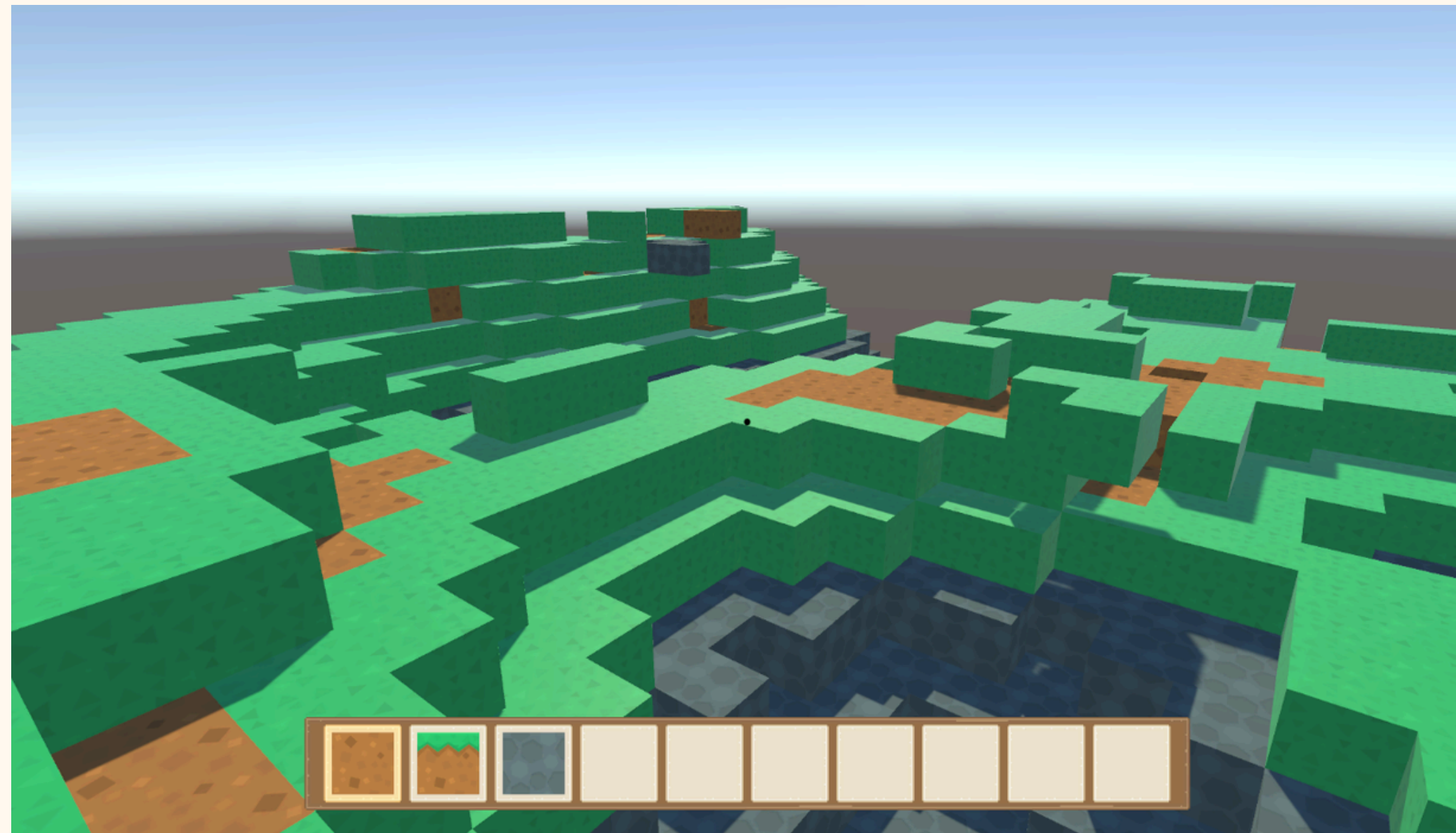
Deuxième étape : placer l'herbe à la surface

- Dans WorldGenerator.cs, fonction Generate ajouter un check
- le bruit de Perlin calcule la hauteur du sol
- $y == \text{height}$ correspond au bloc de surface
- on met Grass sur ce bloc
- en dessous, on garde Dirt

```
int half = chunkSize / 2;  
  
for (int x = -half; x < half; x++)  
    for (int z = -half; z < half; z++)  
    {  
        float nx = (x + offset.x) * noiseScale;  
        float nz = (z + offset.y) * noiseScale;  
        int height = Mathf.FloorToInt(Mathf.PerlinNoise(nx, nz) * terrainHeight);  
  
        for (int y = 0; y <= height; y++)  
        {  
            BlockType type;  
  
            if (y <= 3){  
                type = BlockType.Stone;  
            }  
            else if (y == height){  
                type = BlockType.Grass;  
            }  
            else{  
                type = BlockType.Dirt;  
            }  
  
            blocks[new Vector3Int(x, y, z)] = type;  
        }  
    }  
  
return blocks;  
}
```



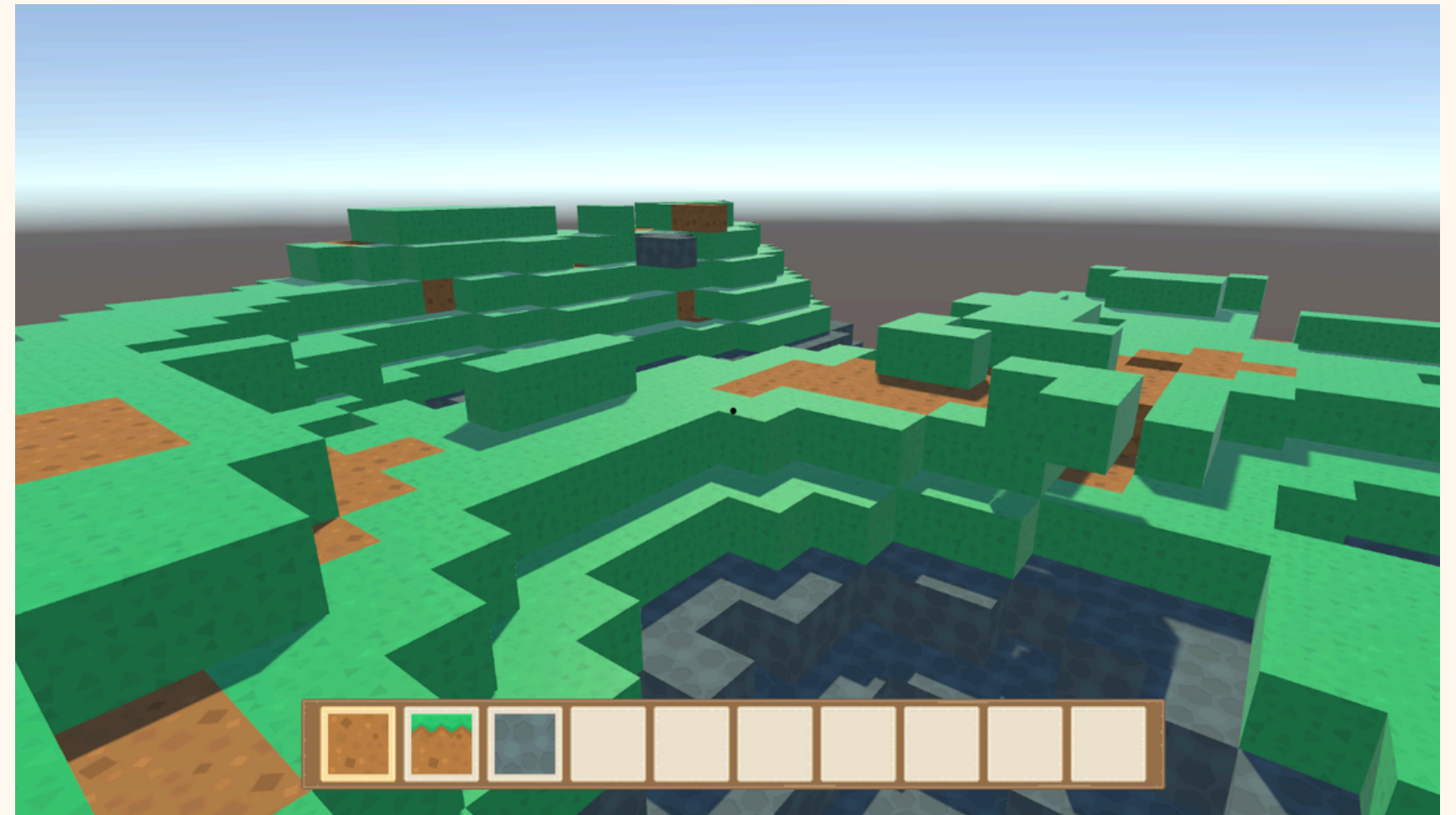
Barre d'inventaire



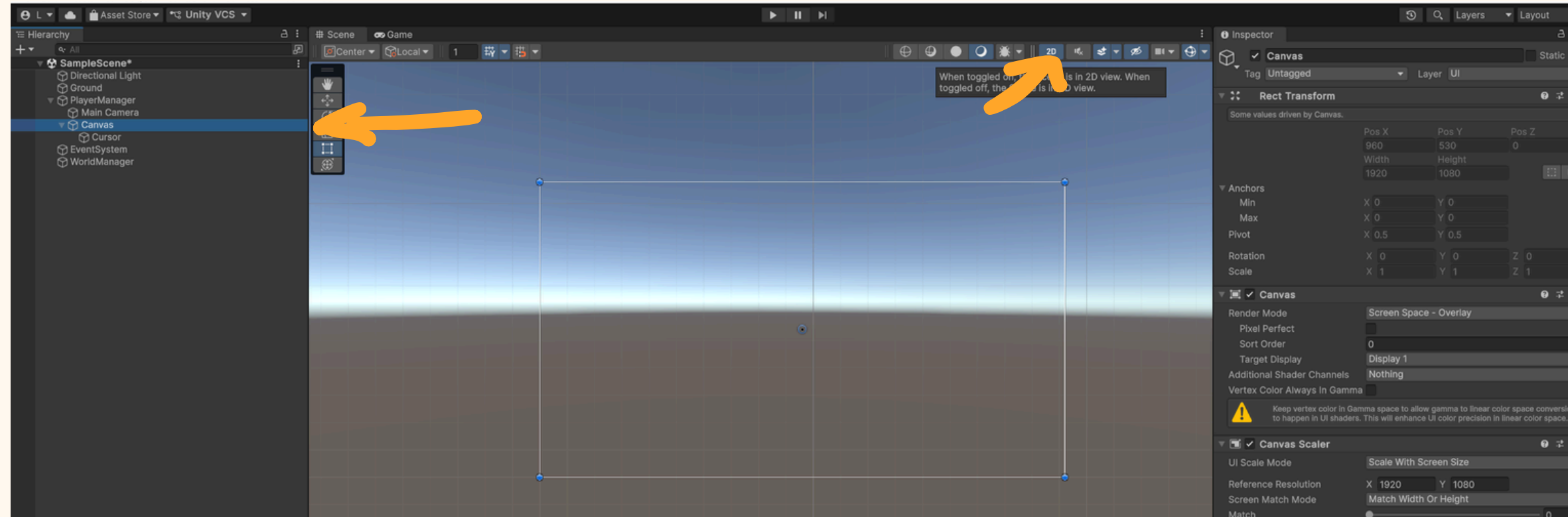
Barre d'inventaire

Ça consiste en quoi ?

- Une image de fond
- Une image pour chaque slot
- Une image pour les blocs



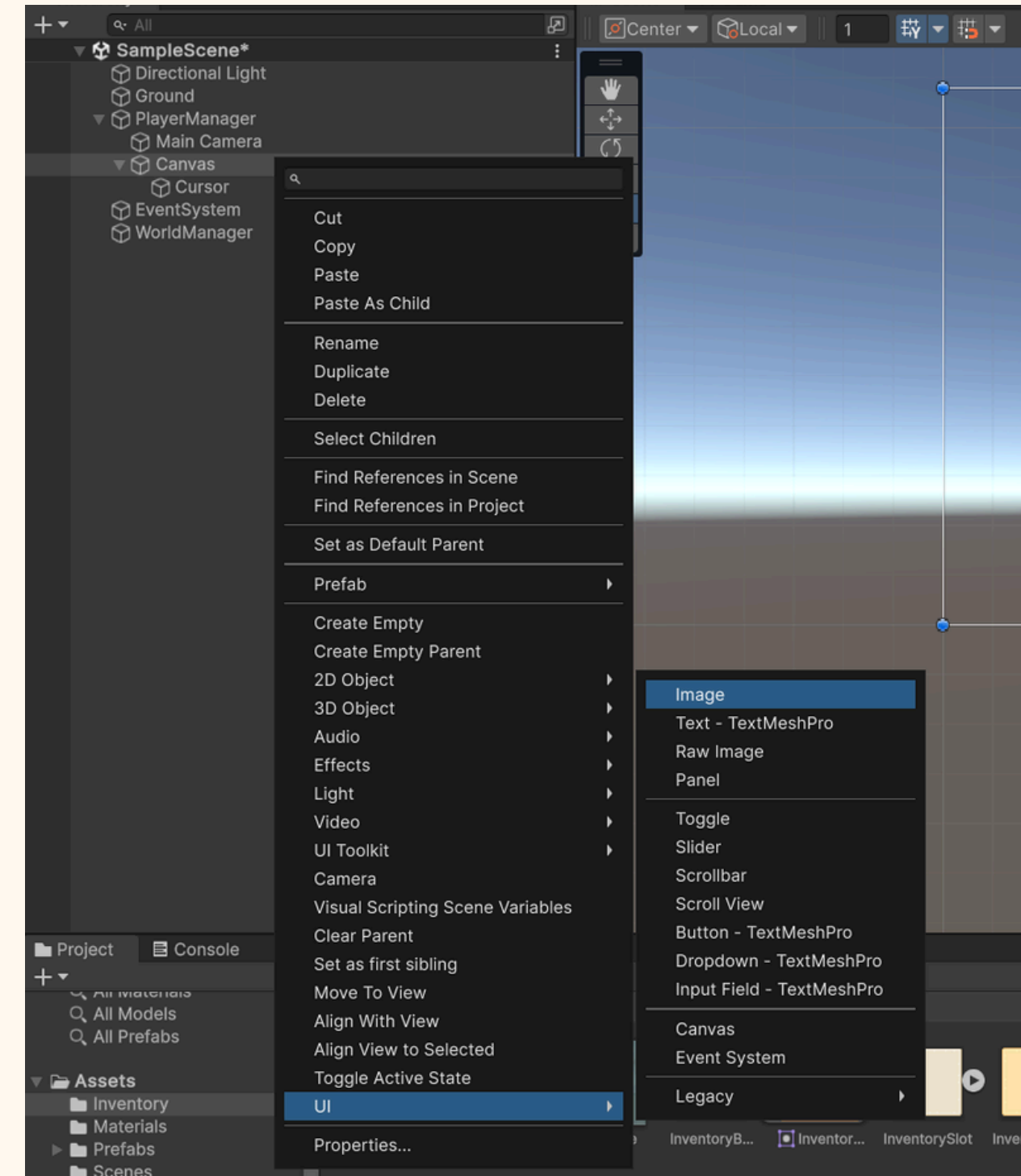
Barre d'inventaire



**Appuyez sur Canevas puis sur 2D. Cela vas changer l'angle de vue.
Ce sera plus facile pour créer l'inventaire.**

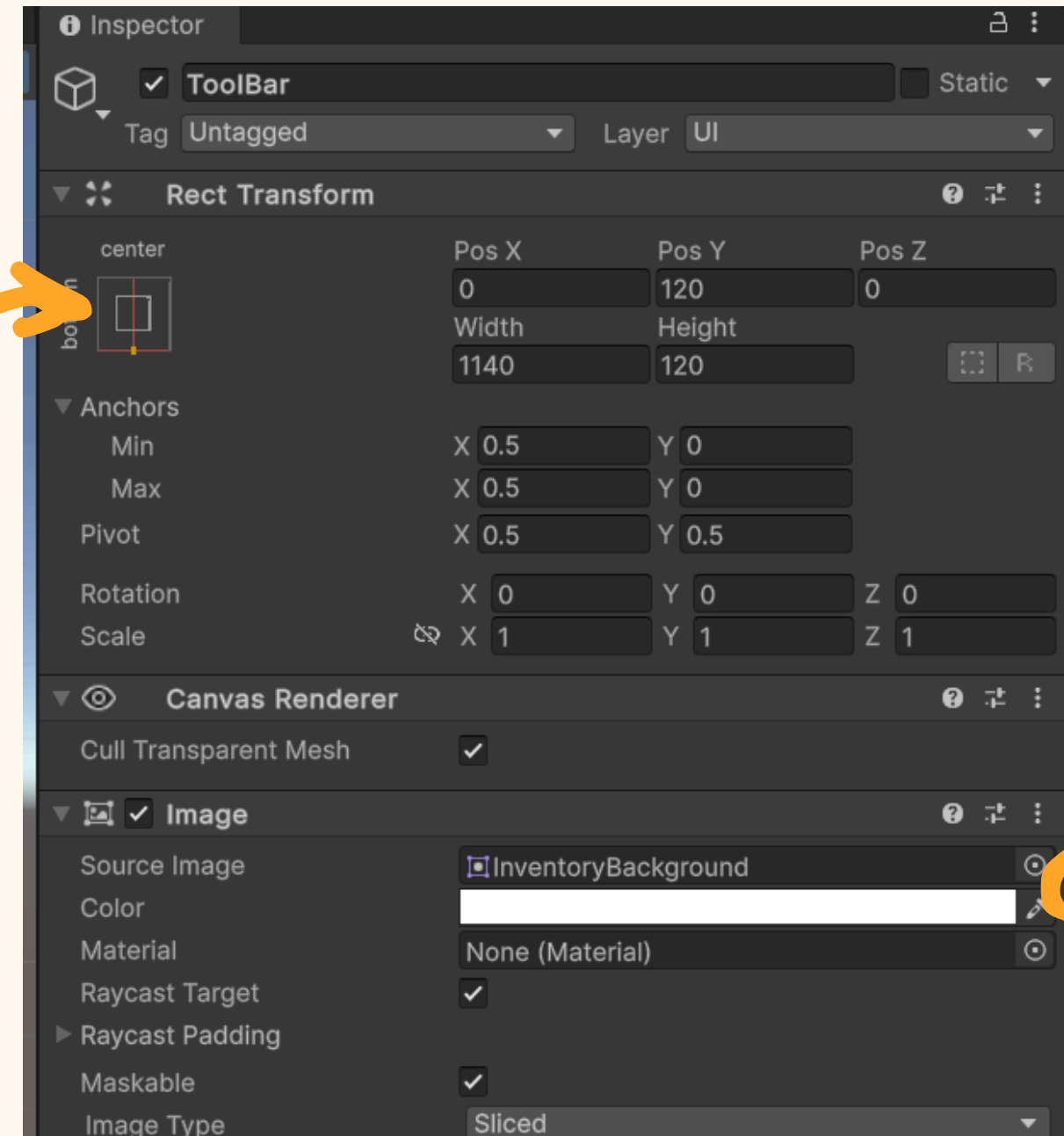
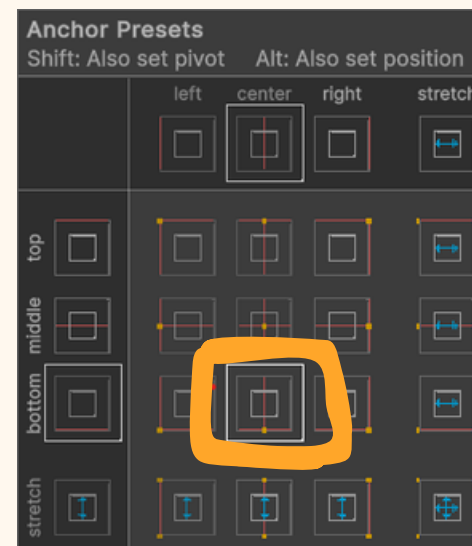
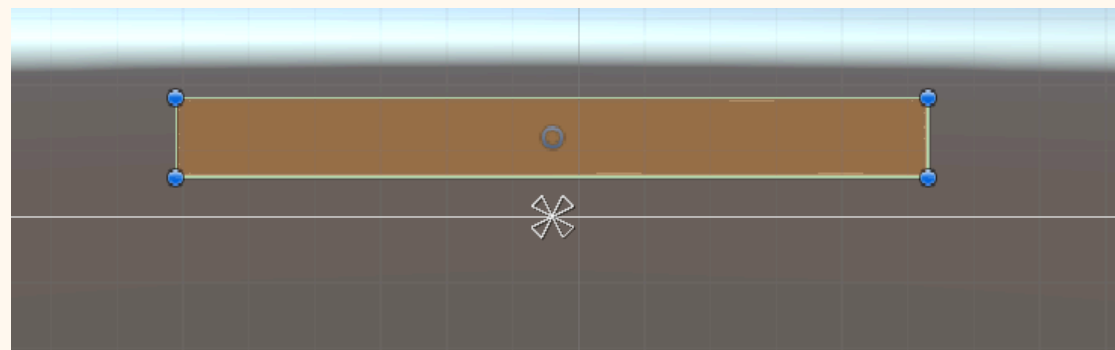
Barre d'inventaire

Ajoutez une Image et appelez la "ToolBar" au Canevas du joueur. Ce sera notre barre d'inventaire. Elle sera parent de toutes nos autres images.



Barre d'inventaire

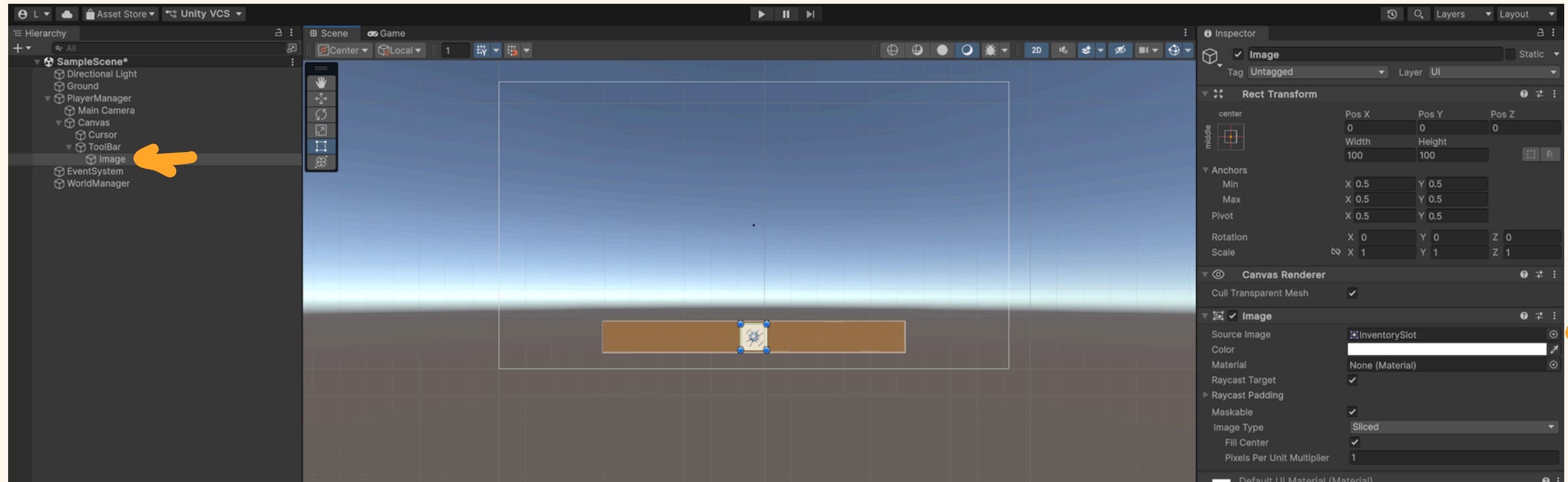
Placez l'image comme indiqué et sélectionnez l'enchrage en "Bottom Center"



L'anchrage permet de définir où est le point de départ de notre image.

Sélectionnez l'image de fond

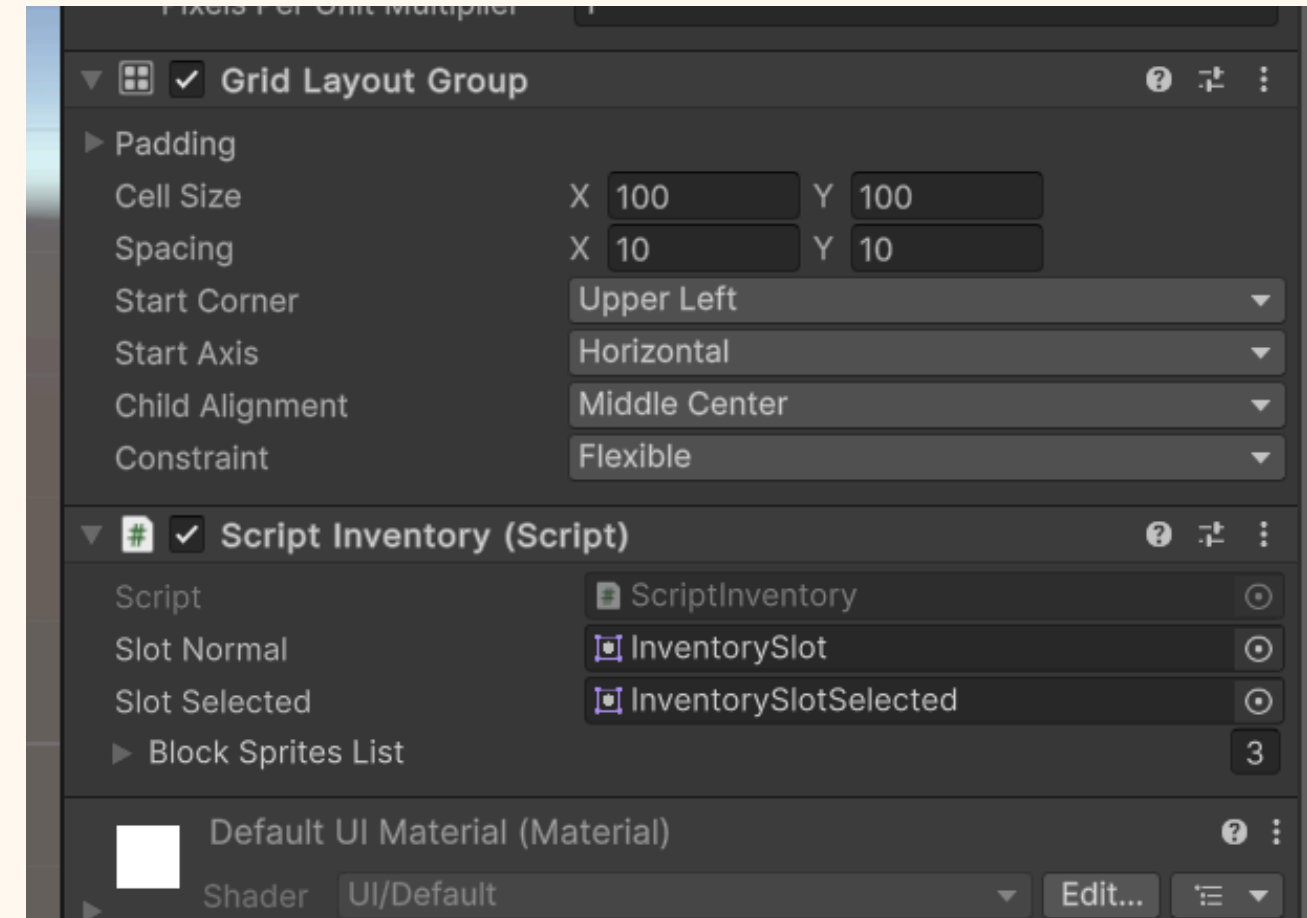
Barre d'inventaire



Créez une Image dans votre ToolBar et sélectionnez l'image "Inventory Slot". Cette image vas définir un des 10 slots de notre barre

Barre d'inventaire

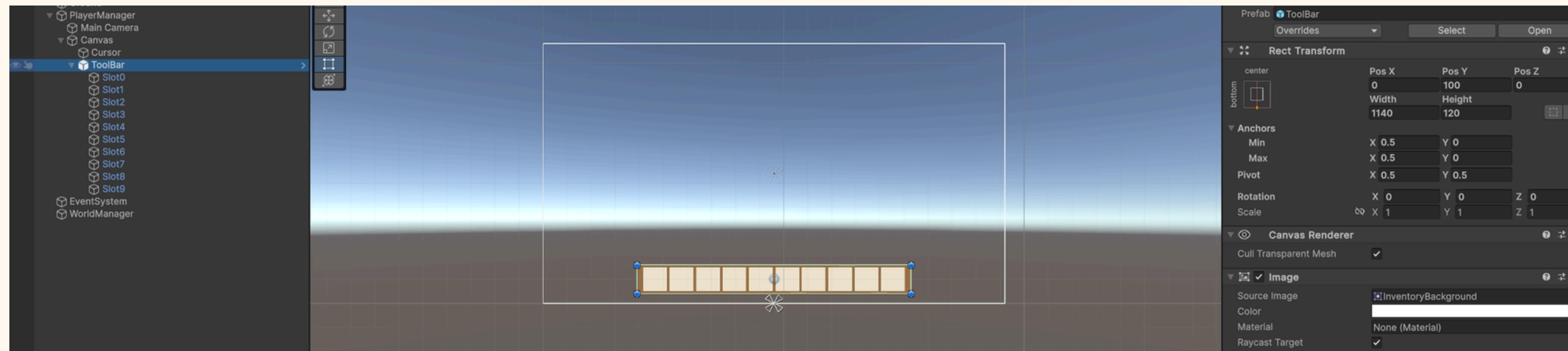
Appliquez un "Grid Layout Group" à votre ToolBar. Pas à votre Slot!



Un Grid Layout permet d'espacer tout les enfants de l'objet sur une grille.

Ici nous allons faire une ou chaque objet fait 100 x 100, comme notre slot, et avec 10 d'ecart entre chaque cellule. On choisit l'alignement au Milieu Centre.

Barre d'inventaire



Dupliquez et rennomez vos slots, ils se mettront en place tout seuls !

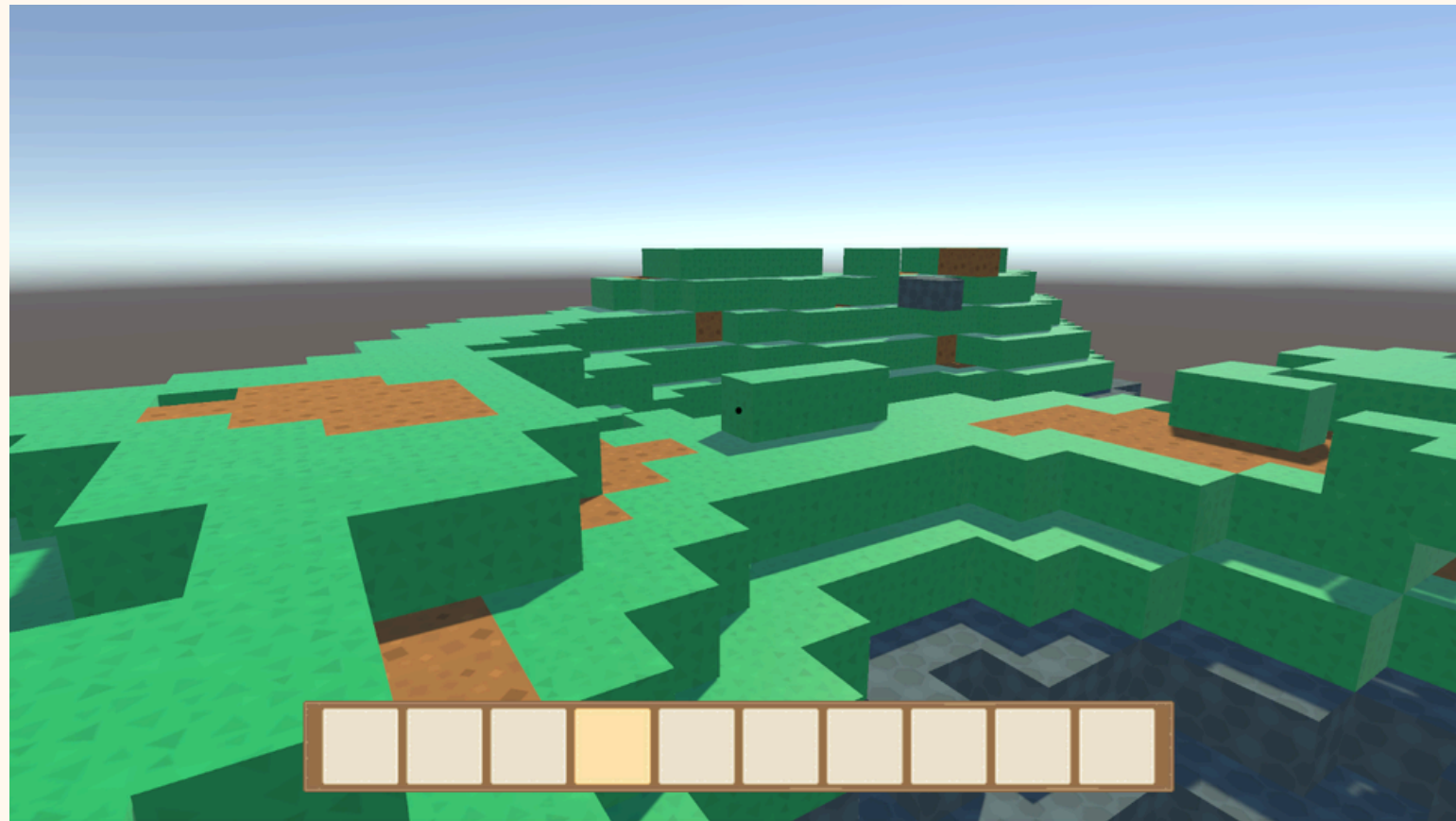
Barre d'inventaire

**Maintenant il vas falloir gérer cette barre
avec un Script.**

**Créez un nouveau Script "ScriptInventory" s'il
n'existe pas encore.**

Barre d'inventaire

Nous allons commencer par la sélection du slot avec la molette



Pour cela, nous pouvons simplement remplacer l'image du slot par une image plus marquée à l'endroit voulu

Barre d'inventaire

**Pour cela nous avons besoin des deux Sprites
(des images transformée pour être affichée
sur un Canevas)**

**Il nous faut aussi une liste qui contient tout
les images de nos slots. On parle bien des
Objets "Slots" qui sont des images dans
Toolbar.**

Et une variable pour notre sélection de slot.

```
public class ScriptInventory : MonoBehaviour
{
    public Sprite slotNormal;
    public Sprite slotSelected;

    private List<Image> slots = new();
    private int selectedSlot = 0;

    void Start()
```

Barre d'inventaire

Cette fonction permet de mettre à jour les images des slots. On vas l'appeler dans d'autres fonctions

```
void UpdateHotbarUI()
{
    for (int i = 0; i < slots.Count; i++)
    {
        if (i == selectedSlot)
            slots[i].sprite = slotSelected;
        else
            slots[i].sprite = slotNormal;
    }
}
```

On vas boucler sur chaque slot, et si i (le numéro de notre slot) est égal à selectedSlot, alors on mets sons sprite à slotSelected, et si pas, alors au sprite normal.

Barre d'inventaire

**On boucle sur tout les objets
Slots de Toolbar et on les ajoute
dans notre liste si ils existent.**

**Comme ça nous avons tous les
slots dans l'ordre**

```
void Start()
{
    foreach (Transform child in transform)
    {
        Image img = child.GetComponent<Image>();
        if (img != null)
            slots.Add(img);
    }

    UpdateHotbarUI();
}
```

Barre d'inventaire

On récupère la valeur de notre molette et si elle ne vaut pas zéro :

```
void Update()
{
    float scroll = Input.GetAxis("Mouse ScrollWheel");

    if (scroll != 0f)
    {
        int direction = scroll > 0 ? -1 : 1;
        selectedSlot = (selectedSlot + direction + slots.Count) % slots.Count;

        UpdateHotbarUI();
    }
}
```

On vas récupérer la direction de la souris (vers le haut ou bas) en regardant si scroll est positif ou négatif. En fonction de ça, direction vaut -1 ou 1.

Barre d'inventaire

**On ajoute ensuite +1 ou -1 à
notre selectedSlot**

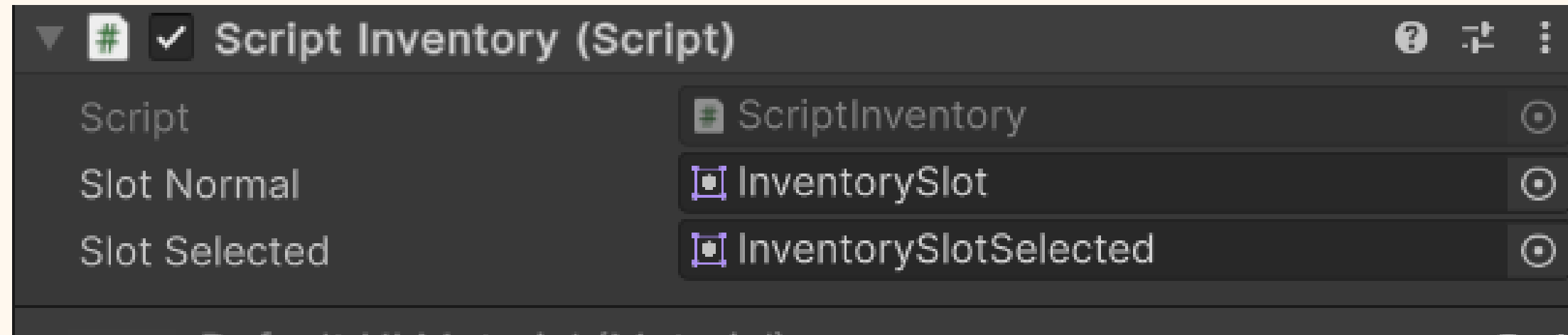
```
void Update()
{
    float scroll = Input.GetAxis("Mouse ScrollWheel");

    if (scroll != 0f)
    {
        int direction = scroll > 0 ? -1 : 1;
        selectedSlot = (selectedSlot + direction + slots.Count) % slots.Count;

        UpdateHotbarUI();
    }
}
```

**On utilise aussi modulo % (qui équivaut au reste de la division)
pour ne pas déborder. Si on arrive à 9 +1, on revient à 0**

Barre d'inventaire



Glissez le script sur ToolBar et glissez les sprites qui sont dans Inventory

Parfait ! maintenant il faut réussir à changer de bloc et voir lequel on a choisi

Barre d'inventaire

```
private List<BlockType> hotbar_slots = new()
{
    BlockType.Dirt,
    BlockType.Grass,
    BlockType.Stone,
};

[System.Serializable]
public class BlockSpriteEntry
{
    public BlockType type;
    public Sprite sprite;
}

public List<BlockSpriteEntry> blockSpritesList = new();
private Dictionary<BlockType, Sprite> blockSprites;

private PlayerAction playerAction;
```

Parfait ! Maintenant il faut réussir à changer de bloc et voir lequel on a choisi

Barre d'inventaire

```
private List<BlockType> hotbar_slots = new()
{
    BlockType.Dirt,
    BlockType.Grass,
    BlockType.Stone,
};

[System.Serializable]
public class BlockSpriteEntry
{
    public BlockType type;
    public Sprite sprite;
}

public List<BlockSpriteEntry> blockSpritesList = new();
private Dictionary<BlockType, Sprite> blockSprites;

private PlayerAction playerAction;
```

Ici c'est un peu compliqué, la hotbar_slots est notre inventaire (nos blocs)

Et la liste et le dictionnaire servent à lier et récupérer un type de bloc et une image liée.

On a aussi besoin de PlayerAction, qui est notre script qui pose des blocs, pour changer de bloc à poser :)

Barre d'inventaire

```
playerAction = FindObjectOfType<PlayerAction>();

// Crée un dictionnaire pour accéder rapidement aux sprites par type de bloc

blockSprites = new Dictionary<BlockType, Sprite>();

foreach (var entry in blockSpritesList)
{
    blockSprites[entry.type] = entry.sprite;
}
```

Dans Void Start, on récupère le PlayerAction comme dit plus haut, puis on va compléter notre dictionnaire en liant les BlockType aux sprites. On a déclaré les variables à la slide d'avant, et ici on leur donne des valeurs.

Barre d'inventaire

```
for (int i = 0; i < hotbar_slots.Count; i++)
{
    BlockType type = hotbar_slots[i];

    // Si on a un sprite pour ce type de bloc, on l'affiche dans le slot correspondant
    if (blockSprites.TryGetValue(type, out Sprite sprite))
    {
        // Crée une Image enfant pour afficher le sprite du bloc
        GameObject blockImgGO = new GameObject("BlockImage");
        blockImgGO.transform.SetParent(slots[i].transform);
        blockImgGO.transform.localPosition = Vector3.zero;
        blockImgGO.transform.localScale = new Vector3(0.8f, 0.8f, 1f); // Ajuste la taille du sprite

        Image blockImg = blockImgGO.AddComponent<Image>();
        blockImg.sprite = sprite;
    }
}

playerAction.ChangeBlockType(hotbar_slots[0]);
```

- Toujours dans Void Start :
- On boucle sur nos blocs dans l'inventaire
- Si ils ont un sprite alors :
- On crée une nouvelle image sur notre slot avec comme parent le slot.
- On ajuste sa taille et sa position
- Et on fini par mettre l'image du bloc en question

Par défaut le slot sélectionné est le premier.

Barre d'inventaire

Pour changer d'objet à placer, nous faisons appel au Player action. On vérifie en premier si notre slot est bien sur un slot valide. Puis on fait appel à ChangeBlockType

```
void updatePlayerAction()
{
    if (selectedSlot < hotbar_slots.Count)
        playerAction.ChangeBlockType(hotbar_slots[selectedSlot]);
}
```

Il faut aussi l'ajouter à notre script PlayerAction

```
public void ChangeBlockType(BlockType newBlockType)
{
    selectedBlock = newBlockType;
}
```


Barre d'inventaire

```
void Update()
{
    float scroll = Input.GetAxis("Mouse ScrollWheel");

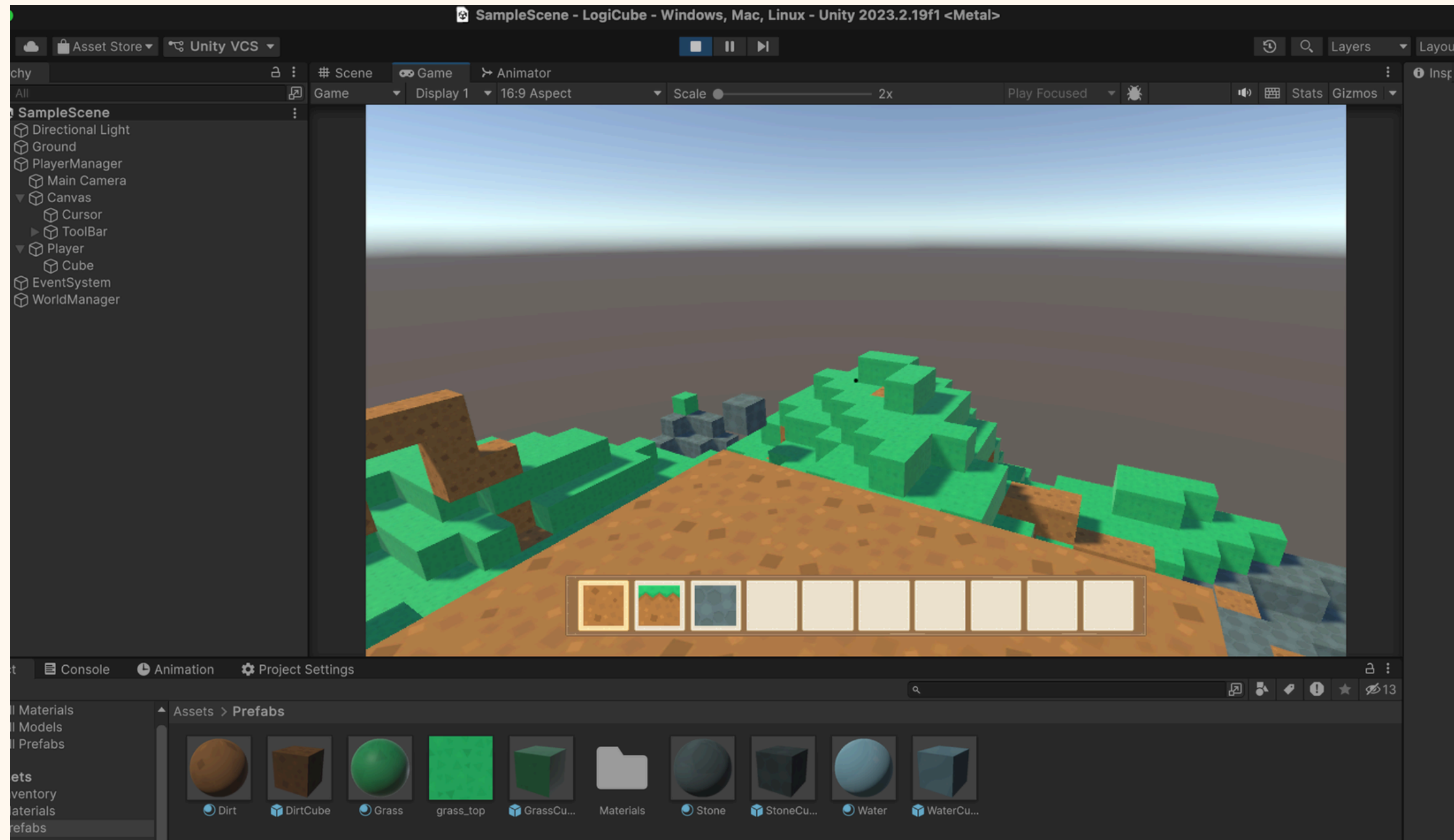
    if (scroll != 0f)
    {
        int direction = scroll > 0 ? -1 : 1;
        selectedSlot = (selectedSlot + direction + slots.Count) % slots.Count;

        UpdateHotbarUI();
        updatePlayerAction();
    }
}
```

Et pour finir, ajoutez updatePlayerAction au void Updtae pour qu'il soit appelé quand on change de slot avec la souris !

Personnage

Objectif : créer un joueur capable de marcher sur notre mesh



Première étape : ajouter un collider à notre mesh

```
private void RebuildChunk()
{
    var visibleFaces = new Dictionary<Vector3Int, List<FaceDirection>>();

    foreach (var (pos, type) in blocks)
    {
        if (!type.IsSolid()) continue;

        var faces = FaceCuller.GetVisibleFaces(pos, blocks);
        if (faces.Count > 0)
            visibleFaces[pos] = faces;
    }

    Mesh mesh = ChunkMeshBuilder.Build(visibleFaces, blocks, out var subMeshOrder);

    // Create chunk object once
    if (chunkGO == null)
    {
        chunkGO = new GameObject("Chunk");
        chunkGO.transform.SetParent(transform);

        chunkGO.AddComponent<MeshFilter>();
        chunkGO.AddComponent<MeshRenderer>();
        chunkGO.AddComponent<MeshCollider>();
    }

    // Assign mesh
    chunkGO.GetComponent<MeshFilter>().mesh = mesh;
}
```

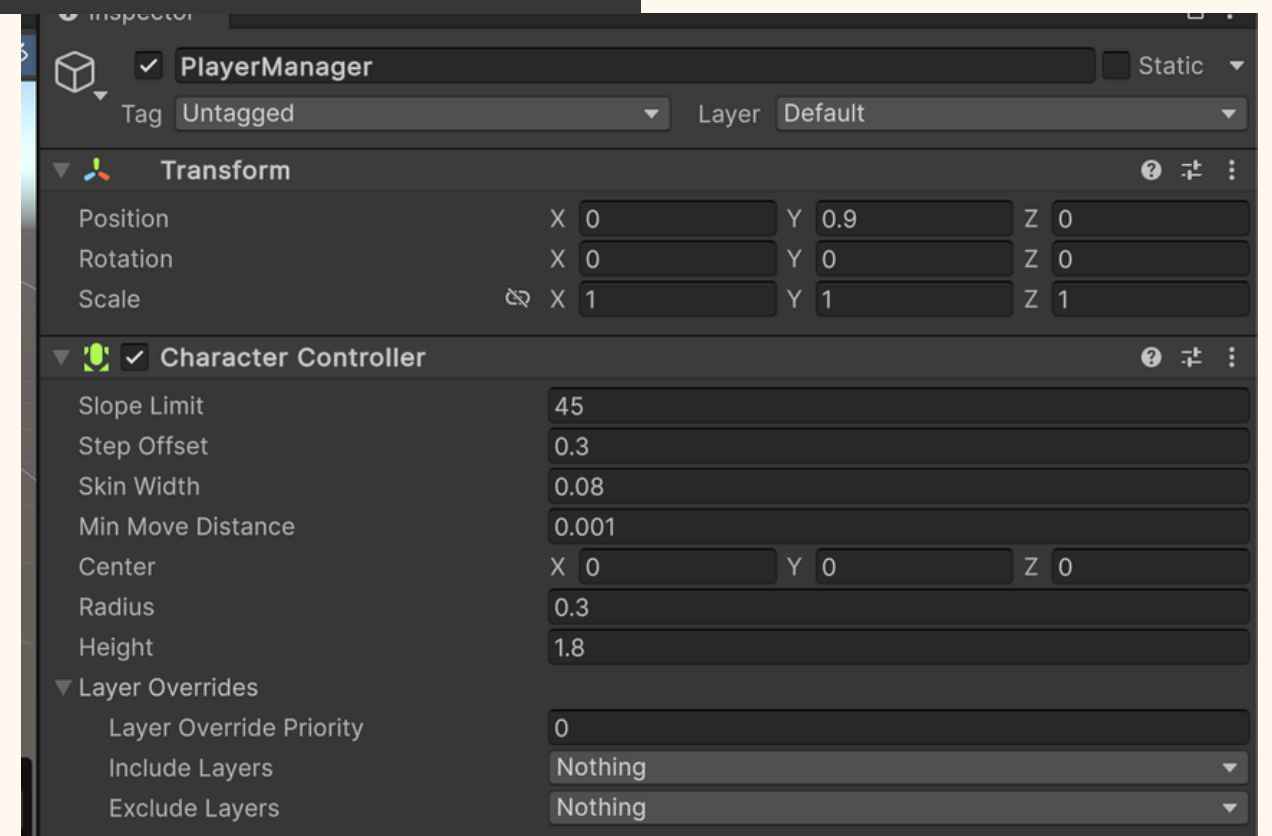
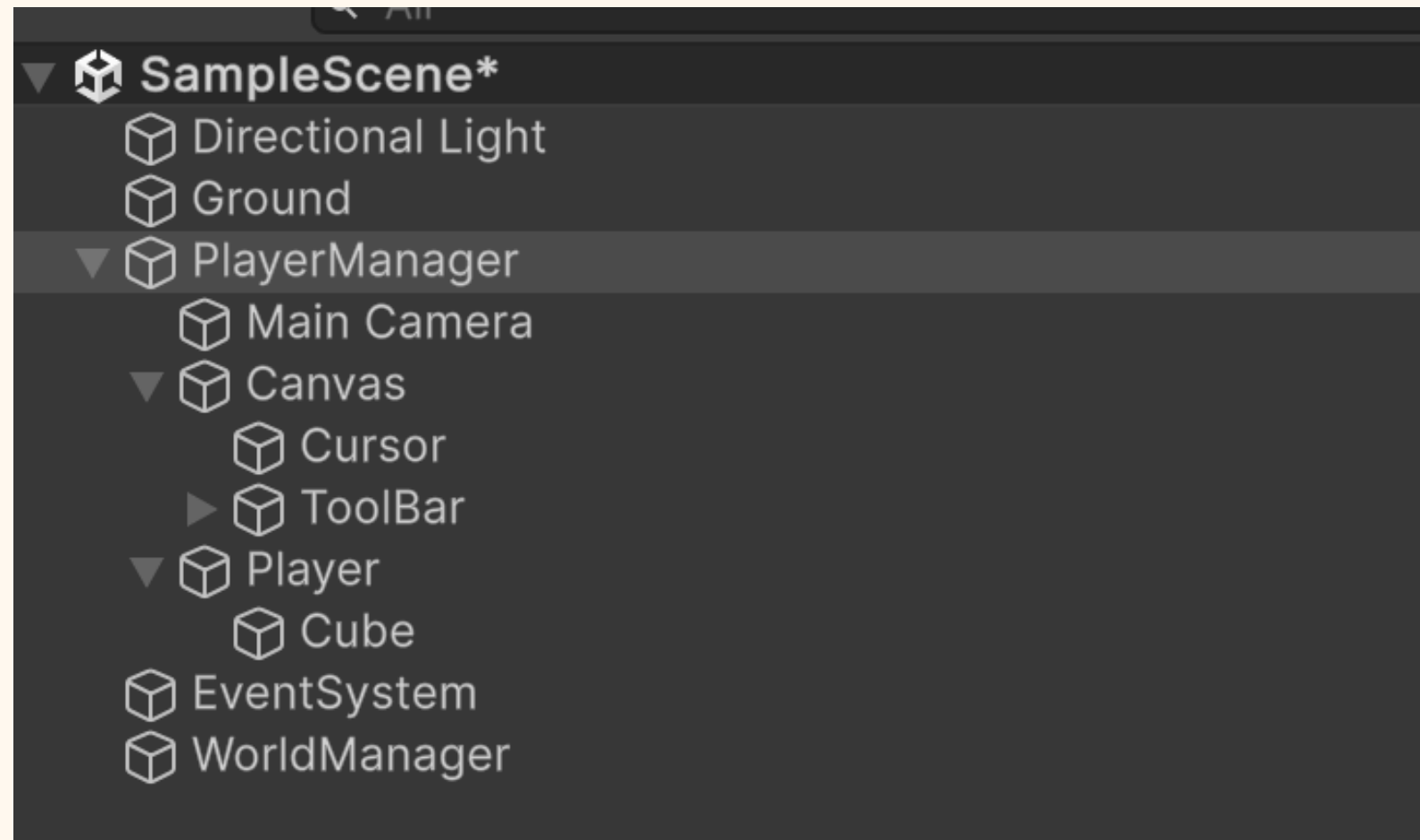
- Dans le script WorldManagerScript, dans la fonction RebuildChunk, on ajoute ce code pour attacher un MeshCollider, un MeshRenderer et un MeshFilter au GameObject du chunk.

Configuration de PlayerManager

Sur PlayerManager, ajoutez un CharacterController.

Réglages conseillés :

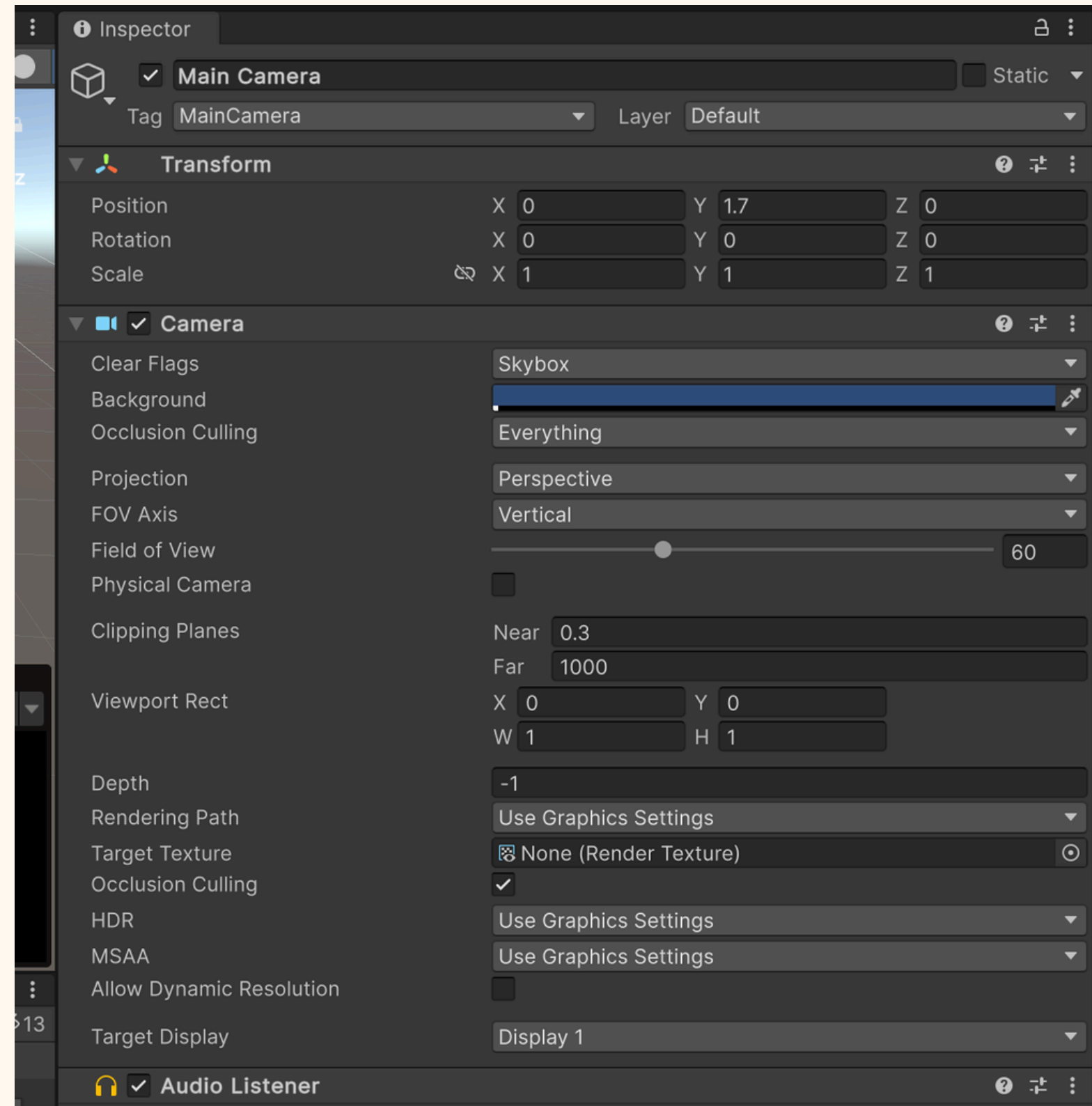
- Center = (0, 0.9, 0)
- Height = 1.8
- Radius = 0.3
- Step Offset = 0.3



Configuration de la caméra

On place la Main Camera comme enfant de PlayerManager pour qu'elle suive le joueur.
Sa position est :

- $X = 0$
- $Y = 1.62$
- $Z = 0$



Remplacer FlyCamera par un script de joueur

- FlyCamera n'est plus adapté à notre joueur
- on le remplace par un script FirstPersonPlayer
- ce script est ajouté à PlayerManager
- il contient les variables utiles pour marcher, sauter et regarder autour
- ajoutez ces variables d'instances

```
public class FirstPersonPlayer : MonoBehaviour
{
    [Header("Déplacements")]
    public float moveSpeed = 5f;           // vitesse pour marcher
    public float flySpeed = 7f;           // vitesse pour voler
    public float lookSpeed = 2f;          // vitesse de la souris
    public float gravity = -20f;          // force qui fait tomber
    public float jumpHeight = 1.2f;       // hauteur du saut
    public float doubleTapDelay = 0.3f;   // temps max entre 2 appuis sur espace

    [Header("Références")]
    public Transform cameraTransform;     // la caméra du joueur

    private CharacterController controller;

    private float pitch = 0f;             // rotation verticale de la caméra
    private float verticalVelocity = 0f;   // vitesse vers le haut ou vers le bas

    private bool isFlying = false;        // est-ce que le joueur vole ?
    private float lastSpacePressTime = -10f; // moment du dernier appui sur espace
```

FirstPersonPlayer : fonctions principales

FirstPersonPlayer : fonctions essentielles

- Look() gère la rotation du joueur et de la caméra
- HandleJumpAndFlyToggle() gère le saut et le mode vol
- le script complet peut être consulté sur GitHub

lien vers le script:

<https://github.com/Ayly-EXE/LogiCube/blob/Cours4/PlayerInventory/Assets/Scripts/FirstPersonPlayer.cs>

```
void Look()
{
    // On lit le mouvement de la souris
    float mouseX = Input.GetAxis("Mouse X") * lookSpeed;
    float mouseY = Input.GetAxis("Mouse Y") * lookSpeed;

    // Tourner à gauche/droite = tourner le joueur entier
    transform.Rotate(0f, mouseX, 0f, Space.Self);

    // On enlève tout penchement bizarre du corps
    Vector3 bodyEuler = transform.eulerAngles;
    transform.eulerAngles = new Vector3(0f, bodyEuler.y, 0f);

    // Regarder en haut/bas = tourner seulement la caméra
    pitch -= mouseY;
    pitch = Mathf.Clamp(pitch, -89f, 89f); // limite pour ne pas tourner complètement
    cameraTransform.localRotation = Quaternion.Euler(pitch, 0f, 0f);
}
```

```
void HandleJumpAndFlyToggle()
{
    // Si on n'appuie pas sur espace, on ne fait rien
    if (!Input.GetKeyDown(KeyCode.Space))
        return;

    float now = Time.time;

    // Vérifie si on a appuyé 2 fois rapidement sur espace
    bool isDoubleTap = now - lastSpacePressTime <= doubleTapDelay;
    lastSpacePressTime = now;

    // Si on est dans les airs et qu'on fait un double appui :
    // on active ou désactive le mode vol
    if (!controller.isGrounded && isDoubleTap)
    {
        isFlying = !isFlying;

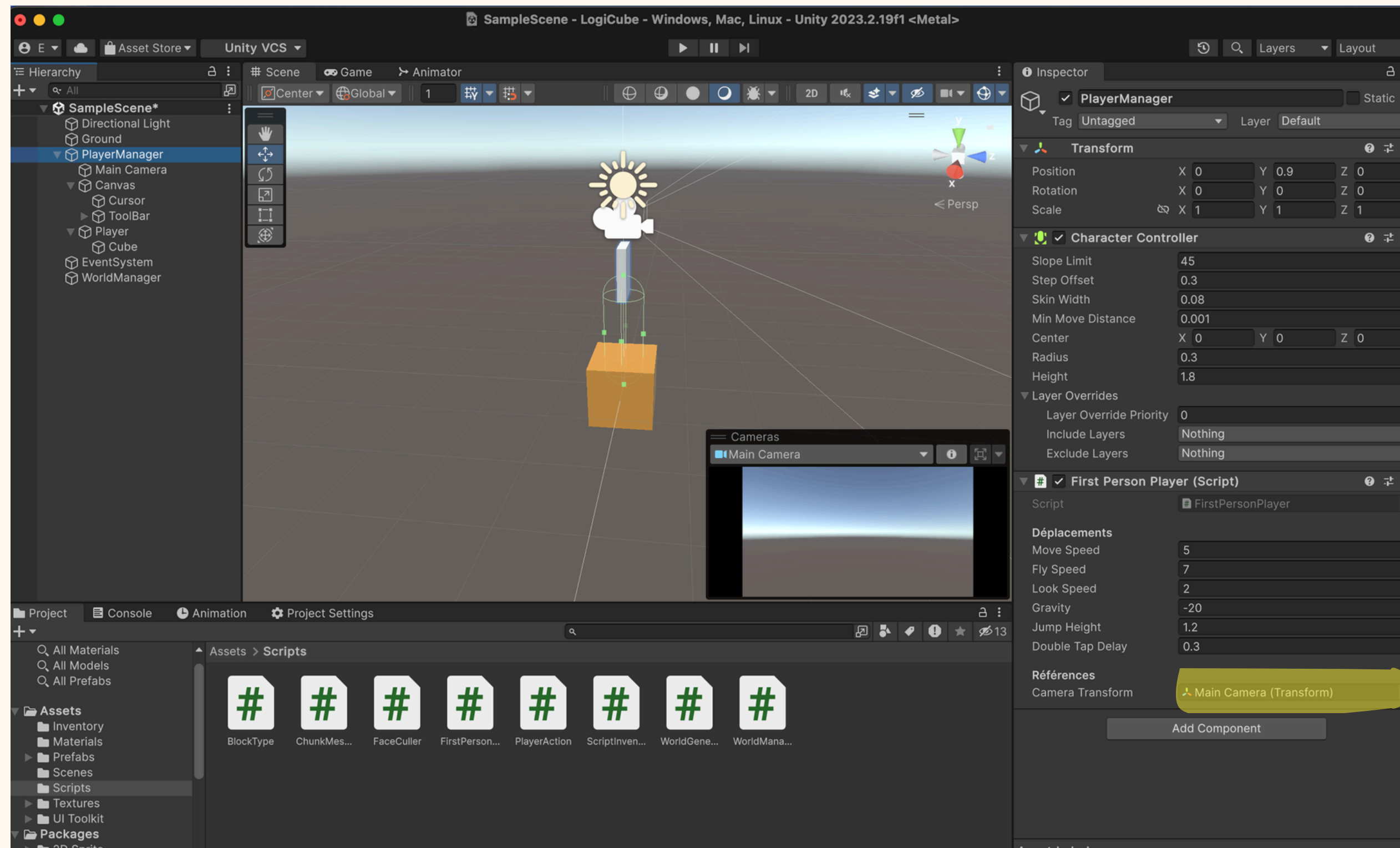
        // Si on commence à voler, on arrête la chute
        if (isFlying)
        {
            verticalVelocity = 0f;
        }

        return;
    }

    // Saut normal : seulement si on est au sol et qu'on ne vole pas
    if (controller.isGrounded && !isFlying)
    {
        verticalVelocity = Mathf.Sqrt(jumpHeight * -2f * gravity);
    }
}
```

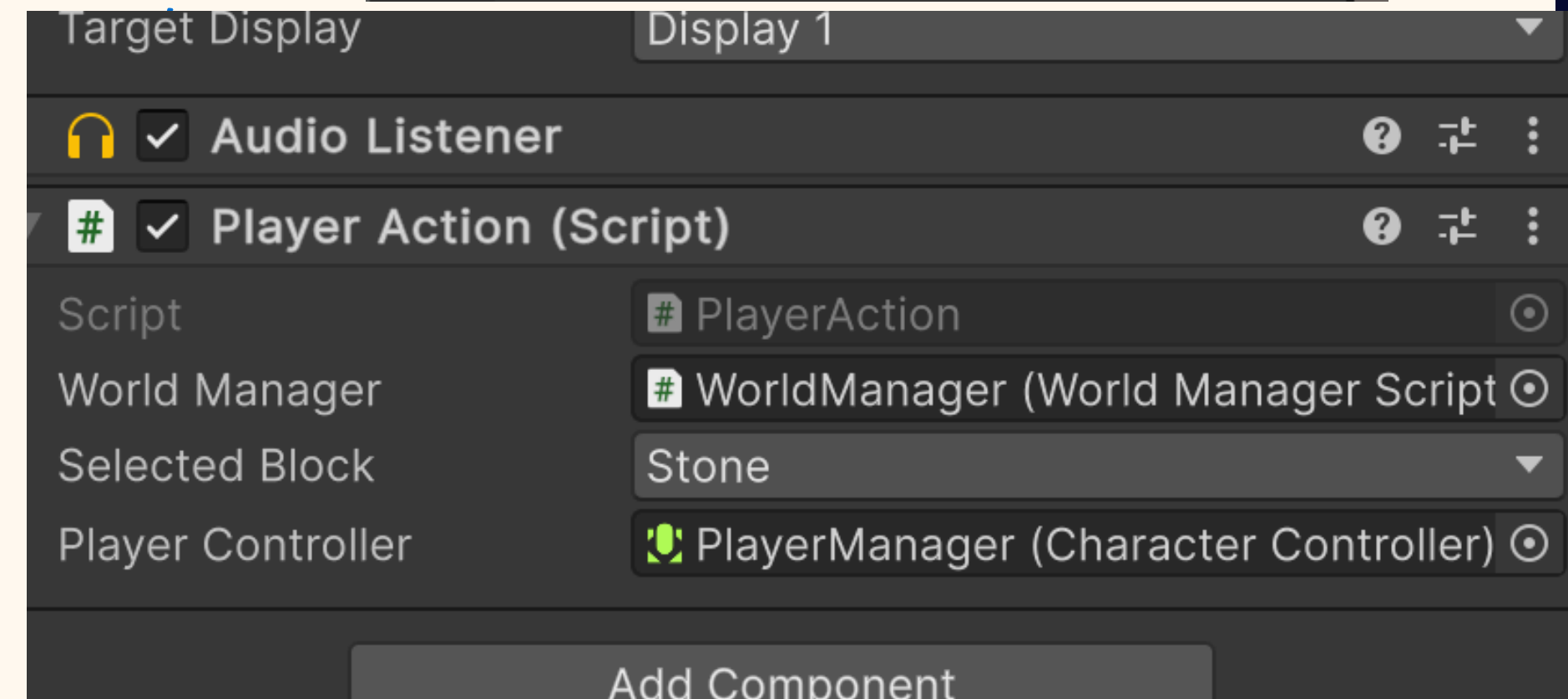
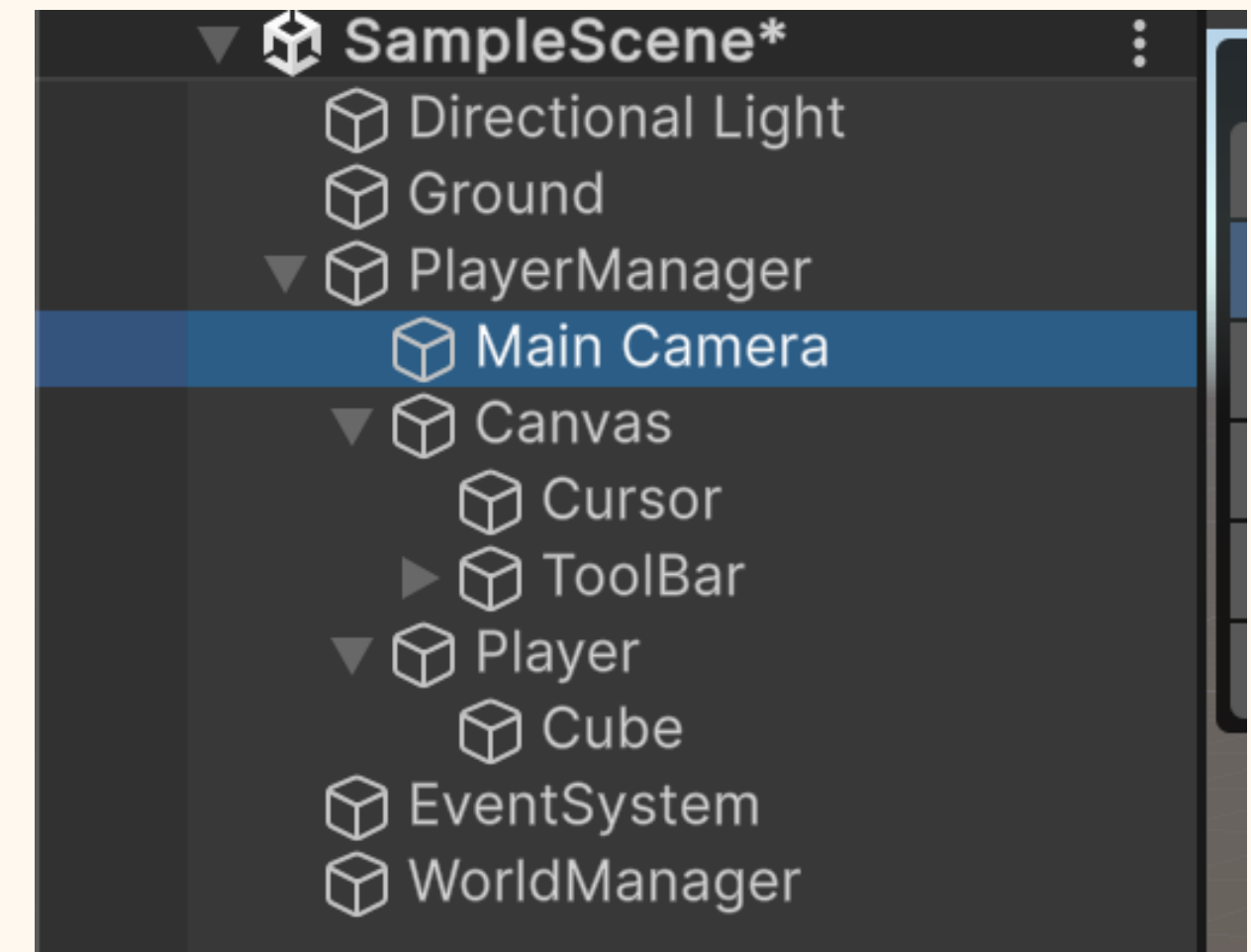

Configurer FirstPersonPlayer dans Unity

- Dans Unity
- sur PlayerManager, ajouter le script FirstPersonPlayer
- glisser la Main Camera dans le champ cameraTransform



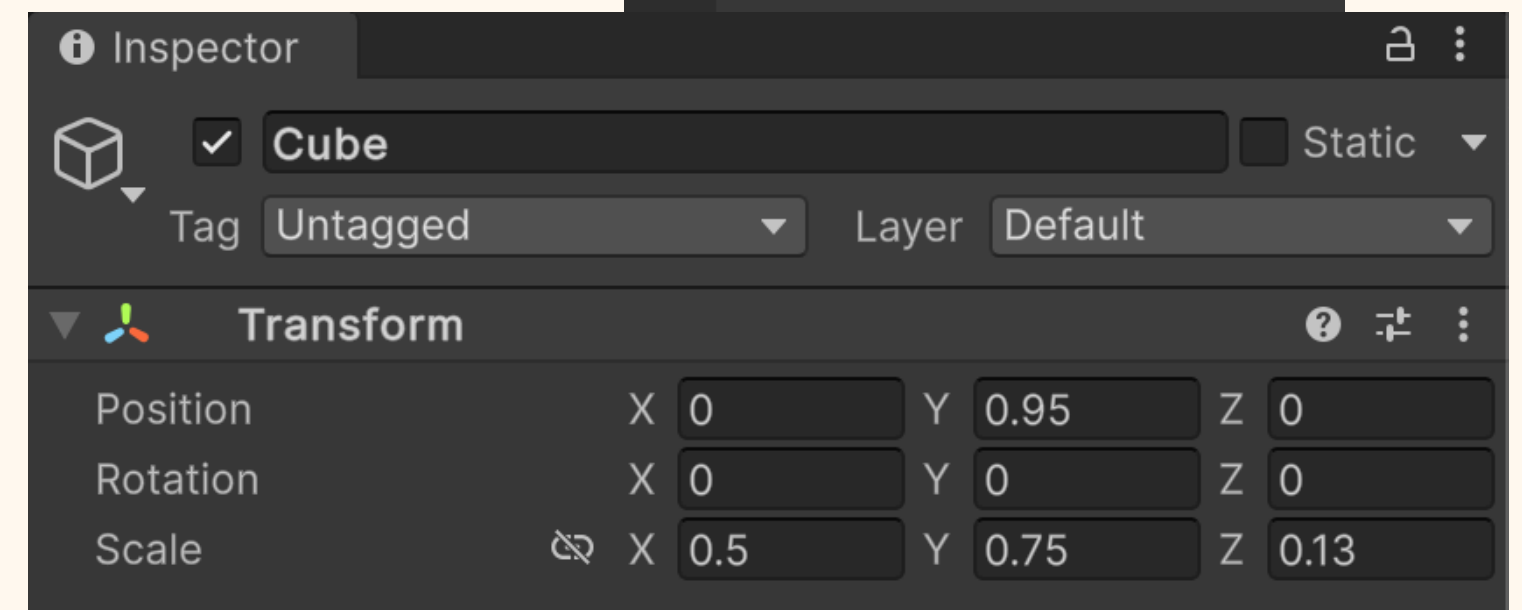
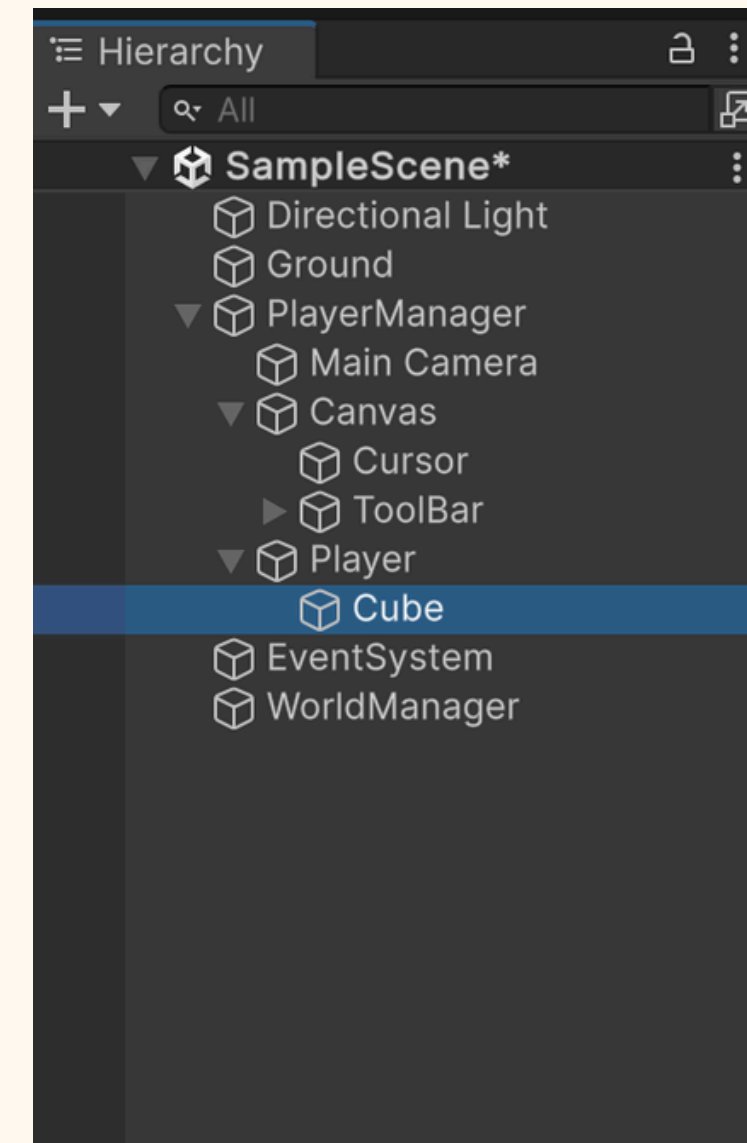
Configurer PlayerAction sur la caméra

- **Ajouter PlayerAction à la Main Camera**
 - dans Unity, sélectionner la Main Camera
 - ajouter le script PlayerAction



Créer un personnage simple

- pour commencer, on utilise un personnage très simple : un cube
- sous **PlayerManager**, créer un objet vide nommé **Player**
- ajouter un **Cube** comme enfant de **Player**
- appliquer les paramètres affichés



Mettre à jour PlayerAction

- Dans PlayerAction.cs :
- clic gauche : casser un bloc
- clic droit : placer un bloc
- rayon depuis la caméra pour viser
- on détecte que le raycast ne touche pas le joueur

```
1 reference
bool IsPlayerObject(Transform hitTransform)
{
    if (playerController == null)
        return false;

    Transform playerRoot = playerController.transform;
    return hitTransform == playerRoot || hitTransform.IsChildOf(playerRoot);
}
```

```
void Update()
{
    if (Input.GetMouseButtonDown(1))
        TryPlaceBlock();

    if (Input.GetMouseButtonDown(0))
        TryBreakBlock();
}
```

```
// Envoie un rayon devant la caméra pour voir ce qu'on vise
2 references
RaycastHit? GetHit(float maxDistance = 100f)
{
    // On démarre un tout petit peu devant la caméra
    // pour éviter de toucher le joueur directement
    Vector3 origin = transform.position + transform.forward * 0.1f;
    Ray ray = new Ray(origin, transform.forward);

    // On récupère tous les objets touchés par le rayon
    RaycastHit[] hits = Physics.RaycastAll(ray, maxDistance);

    // On trie les objets du plus proche au plus loin
    System.Array.Sort(hits, (a, b) => a.distance.CompareTo(b.distance));

    // On cherche le premier objet qui n'est pas le joueur
    foreach (RaycastHit hit in hits)
    {
        if (IsPlayerObject(hit.transform))
            continue;

        return hit;
    }

    // Rien n'a été touché
    return null;
}
```

Mettre à jour PlayerAction

```
1 reference
void TryPlaceBlock()
{
    RaycastHit? hit = GetHit();
    if (!hit.HasValue)
        return;

    Vector3Int placePos = GetBlockPlacementPosition(hit.Value);

    if (!IsInsidePlayer(placePos))
        worldManager.PlaceBlock(placePos, selectedBlock);
}

1 reference
void TryBreakBlock()
{
    RaycastHit? hit = GetHit();
    if (!hit.HasValue)
        return;

    Vector3Int hitBlockPos = GetHitBlockPosition(hit.Value);
    worldManager.DestroyBlock(hitBlockPos);
}
```

```
// Vérifie si le bloc qu'on veut poser serait dans le joueur
bool IsInsidePlayer(Vector3Int blockPos)
{
    if (playerController == null)
        return false;

    // On prend la boîte du joueur
    Bounds b = playerController.bounds;

    // On récupère les coins min et max de cette boîte
    // avec une toute petite marge pour éviter les bords exacts
    Vector3 min = b.min + Vector3.one * 0.001f;
    Vector3 max = b.max - Vector3.one * 0.001f;

    // On parcourt toutes les cases occupées par le joueur
    // entre le coin minimum et le coin maximum
    for (int x = Mathf.FloorToInt(min.x); x <= Mathf.FloorToInt(max.x); x++)
    {
        for (int y = Mathf.FloorToInt(min.y); y <= Mathf.FloorToInt(max.y); y++)
        {
            for (int z = Mathf.FloorToInt(min.z); z <= Mathf.FloorToInt(max.z); z++)
            {
                // Si la case du bloc à poser est une case du joueur,
                // alors on ne peut pas poser le bloc
                if (new Vector3Int(x, y, z) == blockPos)
                    return true;
            }
        }
    }
}
```

script complet:

<https://github.com/Ayly-EXE/LogiCube/blob/Cours4/PlayerInventory/Assets/Scripts/PlayerAction.cs>